Aim:

Design RPC for remote computation where client submits an integer value to the server and server calculates factorial and returns the result to the client program.

Steps:

PDF version of this tutorial: RPC Steps

Create the IDL

Open terminal

```
sudo apt-get update
sudo apt-get install rpcbind
mkdir exp2
cd exp2
gedit fact.x
# add following code in it
struct intpair {
int a;
};
program FACT_PROG {
version FACT_VERS {
int FACT(intpair) = 1;
} = 1;
} = 0x23451111;
# save and exit the file
rpcgen -a -C fact.x
gedit Makefile.fact
# find the following line in the file
CFLAGS += -g
```

and change it to:

CFLAGS += -g **-DRPC_SVC_FG**

find the following line in the same file

RPCGENFLAGS =

and change it to:

RPCGENFLAGS = **-C**

save and exit the file

gedit fact_client.c

we will make some changes in this file (changes are highlighted)

```
#include “fact.h”
void
fact_prog_1(char *host,int a)
{
CLIENT *clnt;
int *result_1;
intpair fact_1_arg;
#ifndefDEBUG
clnt = clnt_create (host, FACT_PROG, FACT_VERS, “udp”);
if (clnt == NULL) {
clnt_pcreateerror (host);
exit (1);
}
#endif /* DEBUG */
fact_1_arg.a=a;
result_1 = fact_1(&fact_1_arg, clnt);
if (result_1 == (int *) NULL) {
clnt_perror (clnt, “call failed”);
}
else
{
printf(“Factorial=%d”,*result_1);
}
#ifndefDEBUG
clnt_destroy (clnt);
#endif /* DEBUG */
}
int
main (int argc, char *argv[])
```

```
{
char *host;
int a,ch;
if (argc < 2) {
printf (“usage: %s server_host\n”, argv[0]);
exit (1);
}
host = argv[1];
do
{
system(“clear”);
printf(“\nEnter a no:: “);
scanf(“%d”,&a);
fact_prog_1 (host,a);
printf(“\nTry again : (1/0) :: “);
scanf(“%d”,&ch);
} while(ch==1);
exit (0);
}
# save and exit the file

gedit fact_server.c
# we will make some changes in this file (changes are highlighted)

#include “fact.h”
int *
fact_1_svc(intpair *argp, struct svc_req *rqstp)
{
static int result,n,fact;
int i;
n=argp->a;
// factorial logic
fact = 1;
printf(“\n Received : n= %d \n”,n);
```

```
for (i=n;i>0;i–)
{
fact=fact * i;
}
result=fact;
return &result;
}
# save and exit the file
# compile
make -f Makefile.fact
# In one terminal, run:
sudo ./fact_server
# In another terminal, run:

./fact_client localhost
```

Design a distributed application using RMI for remote computation where client submits two

strings to the server and server returns the concatenation of the given strings.

Steps:

First install java (if not installed yet)

open terminal and give following commands (one by one)

sudo apt-get update

sudo apt-get install openjdk-7-jre-headless

sudo apt-get install openjdk-7-jdk

find whether ubuntu is 32 bit (i686) or 64 bit (x86_64) by following command

uname -i

Now download eclipse

For 32 bit ubuntu, download eclipse from following link

https://eclipse.org/downloads/download.php?

file=/technology/epp/downloads/release/mars/1/eclipse-jee-mars-1-linux-gtk.tar.gz&mirror_id=466

For 64 bit ubuntu, download eclipse from following link

https://eclipse.org/downloads/download.php?

file=/technology/epp/downloads/release/mars/1/eclipse-jee-mars-1-linux-gtkx86_

64.tar.gz&mirror_id=448

Now Eclipse will be downloaded in Downloads folder, copy-paste it in home folder

Extract it.

After extraction " eclipse" folder will be created.

Just go inside eclipse folder and double click on eclipse file to start the eclipse

In eclipse-> File menu -> new java project -> give project name-> **MyRMI** ->next->finish

Right click on your project -> new->class-> give name-> **Server** -> finish

add following code in that class

```
//RMI Server //
import java.sql.*;
import java.sql.Connection;
import java.rmi.*;
import java.rmi.Naming.*;
import java.rmi.server.*;
import java.rmi.registry.*;
import java.util.Vector;
interface DBInterface extends Remote
{
public String input(String name1,String name2) throws RemoteException;
}
public class Server extends UnicastRemoteObject implements DBInterface
{ int flag=0,n,i,j;
String name3;
ResultSet r;
public Server() throws RemoteException
{ try
{ System.out.println("Initializing Server\nServer Ready");
}
catch (Exception e)
{ System.out.println("ERROR: " +e.getMessage());
}
}
public static void main(String[] args)
{ try
{
```

```
Server rs=new Server();
java.rmi.registry.LocateRegistry.createRegistry(1030).rebind("DBServ",
rs);
}
catch (Exception e)
{
System.out.println("ERROR: " +e.getMessage());
}}
public String input(String name1,String name2)
{
try{
name3=name1.concat(name2);
}
catch (Exception e)
{
System.out.println("ERROR: " +e.getMessage());
}
return name3;
}
}
```

save and exit the file

Right click on your project -> new->class-> give name-> **Client** -> finish

add following code in that class

```
//RMIClient//
import java.sql.*;
import java.rmi.*;
import java.io.*;
import java.util.*;
```

```
import java.util.Vector.*;
import java.lang.*;
import java.rmi.registry.*;
public class Client
{ static String name1,name2,name3;
public static void main(String args[])
{
Client c=new Client();
BufferedReader b = new BufferedReader(new
InputStreamReader(System.in));
int ch;
try { Registry r1 = LocateRegistry.getRegistry ( "localhost", 1030);
DBInterface DI=(DBInterface)r1.lookup("DBServ");
do
{
System.out.println("1.send input stings\n2.Display
concatenated string \nEnter ur choice");
ch= Integer.parseInt(b.readLine());
switch(ch)
{
case 1:
System.out.println(" \n Enter first string:");
name1=b.readLine();
System.out.println(" \n Enter second string:");
name2=b.readLine();
name3=DI.input(name1,name2);
break;
case 2:
```

```
//display
System.out.println("\n Concatenated String is : ");
int i=0;
System.out.println(" " +name3+"");
break;
}
}while(ch>0);
}
catch (Exception e)
{
// System.out.println("ERROR: " +e.getMessage());
}
}
}
```

save and exit the file

Now right click on Server.java -> Run as -> java application

output

Initializing Server

Server Ready

Now right click on Client.java -> Run as -> Run configurations-> Check Main class->

If main class is **Server** then click on search and select **Client.**

In the same window there is one tab named "**Arguments**" -> click on it

In Arguments tab goto **Program arguments** and type -> **localhost**

Click on apply

Click on Run

output

1.send input stings

2.Display concatenated string

Enter ur choice

1

Enter first string:

saibaba

Enter second string:

shirdi

1.send input stings

2.Display concatenated string

Enter ur choice

2

Concatenated String is :

saibabashirdi

Problem Statement - Implement Union, Intersection, Complement and Difference operations on fuzzy sets. Also create fuzzy relations by Cartesian product of any two fuzzy sets and perform max-min composition on any two fuzzy relations.

```
import numpy as np

# Function to perform Union operation on fuzzy sets
def fuzzy_union(A, B):
    return np.maximum(A, B)

# Function to perform Intersection operation on fuzzy sets
def fuzzy_intersection(A, B):
    return np.minimum(A, B)

# Function to perform Complement operation on a fuzzy set
def fuzzy_complement(A):
    return 1 - A

# Function to perform Difference operation on fuzzy sets
def fuzzy_difference(A, B):
    return np.maximum(A, 1 - B)

# Function to create fuzzy relation by Cartesian product of two fuzzy sets
def cartesian_product(A, B):
    return np.outer(A, B)

# Function to perform Max-Min composition on two fuzzy relations
def max_min_composition(R, S):
    return np.max(np.minimum.outer(R, S), axis=1)

# Example usage
A = np.array([0.2, 0.4, 0.6, 0.8])  # Fuzzy set A
B = np.array([0.3, 0.5, 0.7, 0.9])  # Fuzzy set B

# Operations on fuzzy sets
union_result = fuzzy_union(A, B)
intersection_result = fuzzy_intersection(A, B)
complement_A = fuzzy_complement(A)
difference_result = fuzzy_difference(A, B)

print("Union:", union_result)
print("Intersection:", intersection_result)
print("Complement of A:", complement_A)
print("Difference:", difference_result)
```

```
Union: [0.3 0.5 0.7 0.9]
```

```
Intersection: [0.2 0.4 0.6 0.8]
Complement of A: [0.8 0.6 0.4 0.2]
Difference: [0.7 0.5 0.6 0.8]
# Fuzzy relations
R = np.array([0.2, 0.5, 0.4])  # Fuzzy relation R
S = np.array([0.6, 0.3, 0.7])  # Fuzzy relation S

# Cartesian product of fuzzy relations
cartesian_result = cartesian_product(R, S)

# Max-Min composition of fuzzy relations
composition_result = max_min_composition(R, S)

print("Cartesian product of R and S:")
print(cartesian_result)

Cartesian product of R and S:
[[0.12 0.06 0.14]
 [0.3  0.15 0.35]
 [0.24 0.12 0.28]]
print("Max-Min composition of R and S:")
print(composition_result)

Max-Min composition of R and S:
[0.2 0.5 0.4]
```

Write code to simulate requests coming from clients and distribute them among the servers using the load balancing algorithms.

```
pip install requests
```

```
import itertools
import requests
import time
import random
import threading
from collections import defaultdict

# Configuration
SERVER_URLS = ["http://localhost:8081", "http://localhost:8082",
"http://localhost:8083"]
NUM_REQUESTS = 15

# --- Load Balancer Implementations ---

class LoadBalancer:
    def __init__(self, servers):
        self.servers = servers
        self.round_robin_cycle = itertools.cycle(servers)
        self.connections = defaultdict(int) # Tracks active connections per
server

    def get_server_round_robin(self):
        """Selects the next server in a cyclic order."""
        return next(self.round_robin_cycle)

    def get_server_least_connections(self):
        """Selects the server with the fewest active connections."""
        # Find the server with the minimum number of active connections
        return min(self.connections, key=self.connections.get)

    def increment_connections(self, server_url):
        """Increments active connections for a server."""
        self.connections[server_url] += 1

    def decrement_connections(self, server_url):
        """Decrements active connections for a server."""
        if self.connections[server_url] > 0:
```

```
            self.connections[server_url] -= 1

# Initialize the load balancer with the servers
lb = LoadBalancer(SERVER_URLS)

# Initialize connections count for least connections algorithm (all start
at 0)
for url in SERVER_URLS:
    lb.connections[url] = 0

# --- Client and Server Simulation Logic ---

def simulate_server_processing(server_url):
    """Simulates a server receiving and processing a request."""
    lb.increment_connections(server_url) # Mark connection as active
    try:
        # In a real scenario, you'd forward the request.
        # For this simulation, we just print the action and wait.
        print(f"-> Routing request to {server_url}. Active connections:
{lb.connections[server_url]}")
        # Simulate variable processing time
        time.sleep(random.uniform(0.1, 0.5))
        print(f"<- Response received from {server_url}")
    finally:
        lb.decrement_connections(server_url) # Mark connection as complete
        print(f"   Connection to {server_url} closed. Active connections:
{lb.connections[server_url]}")


def client_request_simulation(request_id, algorithm="round_robin"):
    """Simulates a client sending a request through the load balancer."""
    if algorithm == "round_robin":
        selected_server = lb.get_server_round_robin()
    elif algorithm == "least_connections":
        selected_server = lb.get_server_least_connections()
    else:
        print(f"Unknown algorithm: {algorithm}")
        return

    print(f"[Client {request_id}] Requesting with {algorithm} to
{selected_server}")
    simulate_server_processing(selected_server)
```

```python
# --- Main Simulation ---

if __name__ == "__main__":
    print(f"Starting load balancing simulation for {NUM_REQUESTS} requests.")
    print("-" * 30)

    # Choose the algorithm to simulate: "round_robin" or "least_connections"
    chosen_algorithm = "round_robin"
    # chosen_algorithm = "least_connections"

    threads = []
    for i in range(NUM_REQUESTS):
        # Create a new thread for each client request to simulate concurrency
        t = threading.Thread(target=client_request_simulation, args=(i, chosen_algorithm))
        threads.append(t)
        t.start()
        # Small delay between requests to make the output clearer
        time.sleep(0.05)

    # Wait for all threads to complete
    for t in threads:
        t.join()

    print("-" * 30)
    print("Simulation complete.")
    print(f"Final connection counts: {lb.connections}")
```

How to Run the Simulation

1. **Save the code:** Save the code above as a Python file (e.g., `load_balancer_simulation.py`).

2. **Run the script:**

```bash
python load_balancer_simulation.py
```

Problem Statement - Optimization of genetic algorithm parameter in hybrid genetic algorithm-neural network modelling: Application to spray drying of coconut milk.

```
import random
from deap import base, creator, tools, algorithms
# Define evaluation function (this is a mock function, replace this with
your actual evaluation function)
def evaluate(individual):
    # Here 'individual' represents the parameters for the neural network
    # You'll need to replace this with your actual evaluation function that
trains the neural network and evaluates its performance
    # Return a fitness value (here, a random number is used as an example)
    return random.random(),
# Define genetic algorithm parameters
POPULATION_SIZE = 10
GENERATIONS = 5
# Create types for fitness and individuals in the genetic algorithm
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)
D:\Python\lib\site-packages\deap\creator.py:185: RuntimeWarning: A class na
med 'FitnessMin' has already been created and it will be overwritten. Consi
der deleting previous creation of that class or rename it.
  warnings.warn("A class named '{0}' has already been created and it "
D:\Python\lib\site-packages\deap\creator.py:185: RuntimeWarning: A class na
med 'Individual' has already been created and it will be overwritten. Consi
der deleting previous creation of that class or rename it.
  warnings.warn("A class named '{0}' has already been created and it "
# Initialize toolbox
toolbox = base.Toolbox()
# Define attributes and individuals
toolbox.register("attr_neurons", random.randint, 1, 100)  # Example: number
of neurons
toolbox.register("attr_layers", random.randint, 1, 5)  # Example: number of
layers
toolbox.register("individual", tools.initCycle, creator.Individual,
(toolbox.attr_neurons, toolbox.attr_layers), n=1)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
# Genetic operators
toolbox.register("evaluate", evaluate)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutUniformInt, low=1, up=100, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)
# Create initial population
population = toolbox.population(n=POPULATION_SIZE)
# Run the genetic algorithm
for gen in range(GENERATIONS):
    offspring = algorithms.varAnd(population, toolbox, cxpb=0.5, mutpb=0.1)

    fitnesses = toolbox.map(toolbox.evaluate, offspring)
```

```
    for ind, fit in zip(offspring, fitnesses):
        ind.fitness.values = fit

    population = toolbox.select(offspring, k=len(population))
# Get the best individual from the final population
best_individual = tools.selBest(population, k=1)[0]
best_params = best_individual
# Print the best parameters found
print("Best Parameters:", best_params)
Best Parameters: [3, 5]
```

Implementation of clonal selection algorithm using python

```
import random

# Fitness function (example: maximize f(x) = x^2)
def fitness(x):
    return x * x

# Initialize population
def initialize_population(size, lower, upper):
    return [random.uniform(lower, upper) for _ in range(size)]

# Clonal selection algorithm
def clonal_selection(pop_size=10, generations=20, clones=5):
    population = initialize_population(pop_size, -10, 10)

    for gen in range(generations):
        # Evaluate fitness
        population = sorted(population, key=fitness, reverse=True)

        # Select best antibodies
        selected = population[:pop_size // 2]

        # Cloning and mutation
        clones_list = []
        for antibody in selected:
            for _ in range(clones):
                mutation = random.uniform(-1, 1)
                clones_list.append(antibody + mutation)
```

```
        # Combine old and new population
        population = selected + clones_list
        population = sorted(population, key=fitness, reverse=True)[:pop_size]

    best_solution = population[0]
    return best_solution, fitness(best_solution)

# Run algorithm
best, best_fitness = clonal_selection()
print("Best Solution:", best)
print("Best Fitness:", best_fitness)
```

Distributed Computing:

Problem Statement - To apply the artificial immune pattern recognition to perform a task of structure damage Classification.

```
import numpy as np
# Generate dummy data for demonstration purposes (replace this with your
actual data)
def generate_dummy_data(samples=100, features=10):
    data = np.random.rand(samples, features)
    labels = np.random.randint(0, 2, size=samples)
    return data, labels
# Define the AIRS algorithm
class AIRS:
    def __init__(self, num_detectors=10, hypermutation_rate=0.1):
        self.num_detectors = num_detectors
        self.hypermutation_rate = hypermutation_rate

    def train(self, X, y):
        self.detectors = X[np.random.choice(len(X), self.num_detectors,
replace=False)]

    def predict(self, X):
        predictions = []
        for sample in X:
            distances = np.linalg.norm(self.detectors - sample, axis=1)
            prediction = int(np.argmin(distances))
            predictions.append(prediction)
        return predictions
# Generate dummy data
data, labels = generate_dummy_data()
# Split data into training and testing sets
split_ratio = 0.8
split_index = int(split_ratio * len(data))
train_data, test_data = data[:split_index], data[split_index:]
train_labels, test_labels = labels[:split_index], labels[split_index:]
# Initialize and train AIRS
airs = AIRS(num_detectors=10, hypermutation_rate=0.1)
airs.train(train_data, train_labels)
# Test AIRS on the test set
predictions = airs.predict(test_data)
# Evaluate accuracy
accuracy = np.mean(predictions == test_labels)
print(f"Accuracy: {accuracy}")
Accuracy: 0.05
```

Problem Statement : Implement DEAP (Distributed Evolutionary Algorithms) using Python

```
import random
from deap import base, creator, tools, algorithms
# Define the evaluation function (minimize a simple mathematical function)
def eval_func(individual):
    # Example evaluation function (minimize a quadratic function)
    return sum(x ** 2 for x in individual),
# DEAP setup
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)
toolbox = base.Toolbox()
# Define attributes and individuals
toolbox.register("attr_float", random.uniform, -5.0, 5.0)  # Example: Float
values between -5 and 5
toolbox.register("individual", tools.initRepeat, creator.Individual,
toolbox.attr_float, n=3)  # Example: 3-dimensional individual
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
# Evaluation function and genetic operators
toolbox.register("evaluate", eval_func)
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)
# Create population
population = toolbox.population(n=50)
# Genetic Algorithm parameters
generations = 20
# Run the algorithm
for gen in range(generations):
    offspring = algorithms.varAnd(population, toolbox, cxpb=0.5, mutpb=0.1)

    fits = toolbox.map(toolbox.evaluate, offspring)
    for fit, ind in zip(fits, offspring):
        ind.fitness.values = fit

    population = toolbox.select(offspring, k=len(population))
# Get the best individual after generations
best_ind = tools.selBest(population, k=1)[0]
best_fitness = best_ind.fitness.values[0]

print("Best individual:", best_ind)
print("Best fitness:", best_fitness)
Best individual: [0.004203628922240612, 0.0011996933224371602, 0.0001696656
6135762735]
Best fitness: 1.9138546620442e-05
```

Implement Ant colony optimization by solving the Traveling salesman problem using python Problem statement- salesman needs to visit a set of cities exactly once and return to the original city. The task is to find the shortest possible route that the salesman can take to visit all the cities and return to the starting city.

```
import numpy as np
import random
# Define the distance matrix (distances between cities)
# Replace this with your distance matrix or generate one based on your
problem
# Example distance matrix (replace this with your actual data)
distance_matrix = np.array([
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
])
# Parameters for Ant Colony Optimization
num_ants = 10
num_iterations = 50
evaporation_rate = 0.5
pheromone_constant = 1.0
heuristic_constant = 1.0
# Initialize pheromone matrix and visibility matrix
num_cities = len(distance_matrix)
pheromone = np.ones((num_cities, num_cities))  # Pheromone matrix
visibility = 1 / distance_matrix  # Visibility matrix (inverse of distance)
C:\Users\UNIQUE\AppData\Local\Temp\ipykernel_19040\3849324448.py:4: Runtime
Warning: divide by zero encountered in divide
  visibility = 1 / distance_matrix  # Visibility matrix (inverse of distanc
e)
# ACO algorithm
for iteration in range(num_iterations):
    ant_routes = []
    for ant in range(num_ants):
        current_city = random.randint(0, num_cities - 1)
        visited_cities = [current_city]
        route = [current_city]

        while len(visited_cities) < num_cities:
            probabilities = []
            for city in range(num_cities):
```

```
                if city not in visited_cities:
                    pheromone_value = pheromone[current_city][city]
                    visibility_value = visibility[current_city][city]
                    probability = (pheromone_value ** pheromone_constant) *
(visibility_value ** heuristic_constant)
                    probabilities.append((city, probability))

            probabilities = sorted(probabilities, key=lambda x: x[1],
reverse=True)
            selected_city = probabilities[0][0]
            route.append(selected_city)
            visited_cities.append(selected_city)
            current_city = selected_city

        ant_routes.append(route)

        # Update pheromone levels
    delta_pheromone = np.zeros((num_cities, num_cities))
    for ant, route in enumerate(ant_routes):
        for i in range(len(route) - 1):
            city_a = route[i]
            city_b = route[i + 1]
            delta_pheromone[city_a][city_b] += 1 /
distance_matrix[city_a][city_b]
            delta_pheromone[city_b][city_a] += 1 /
distance_matrix[city_a][city_b]

    pheromone = (1 - evaporation_rate) * pheromone + delta_pheromone
# Find the best route
best_route_index = np.argmax([sum(distance_matrix[cities[i]][cities[(i + 1)
% num_cities]] for i in range(num_cities)) for cities in ant_routes])
best_route = ant_routes[best_route_index]
shortest_distance = sum(distance_matrix[best_route[i]][best_route[(i + 1) %
num_cities]] for i in range(num_cities))
print("Best route:", best_route)
print("Shortest distance:", shortest_distance)
```

```
Best route: [0, 1, 3, 2]
Shortest distance: 80
```

python code for Design and develop a distributed application to find the coolest/hottest year from the available weather data. Use weather data from the Internet and process it using MapReduce.

Mapper Code (mapper.py)

This extracts **year and temperature** from input data.

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()

    # Skip empty lines
    if not line:
        continue

    # Assuming CSV format: Year,Month,Day,Temperature
    parts = line.split(',')

    try:
        year = parts[0]
        temp = float(parts[3])

        # Emit: year and temperature
        print(f"{year}\t{temp}")

    except:
        continue
```

Reducer Code (reducer.py)

This calculates **average temperature per year**.

```
#!/usr/bin/env python3
import sys

current_year = None
temp_sum = 0
count = 0

for line in sys.stdin:
    line = line.strip()

    year, temp = line.split('\t')
```

```
    temp = float(temp)

    if current_year == year:
        temp_sum += temp
        count += 1
    else:
        if current_year:
            avg_temp = temp_sum / count
            print(f"{current_year}\t{avg_temp}")

        current_year = year
        temp_sum = temp
        count = 1

# Last year output
if current_year:
    avg_temp = temp_sum / count
    print(f"{current_year}\t{avg_temp}")
```

Final Processing Script (Optional)

This finds the **hottest and coolest year** from reducer output.

```
# final.py

hottest_year = None
coolest_year = None
max_temp = float('-inf')
min_temp = float('inf')

with open('output.txt', 'r') as f:
    for line in f:
        year, avg = line.strip().split('\t')
        avg = float(avg)

        if avg > max_temp:
            max_temp = avg
            hottest_year = year

        if avg < min_temp:
            min_temp = avg
            coolest_year = year

print("Hottest Year:", hottest_year, "Avg Temp:", max_temp)
print("Coolest Year:", coolest_year, "Avg Temp:", min_temp)
```

How to Run (Hadoop Streaming)

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming*.jar \
-input /input/weather.csv \
-output /output/weather_result \
-mapper mapper.py \
-reducer reducer.py
```

Input (weather.csv)

```
2001,01,01,25
2001,01,02,27
2002,01,01,30
2002,01,02,28
2003,01,01,22
2003,01,02,24
```

Output

```
2001    26.0
2002    29.0
2003    23.0
```

Final Result:

```
Hottest Year: 2002
Coolest Year: 2003
```
